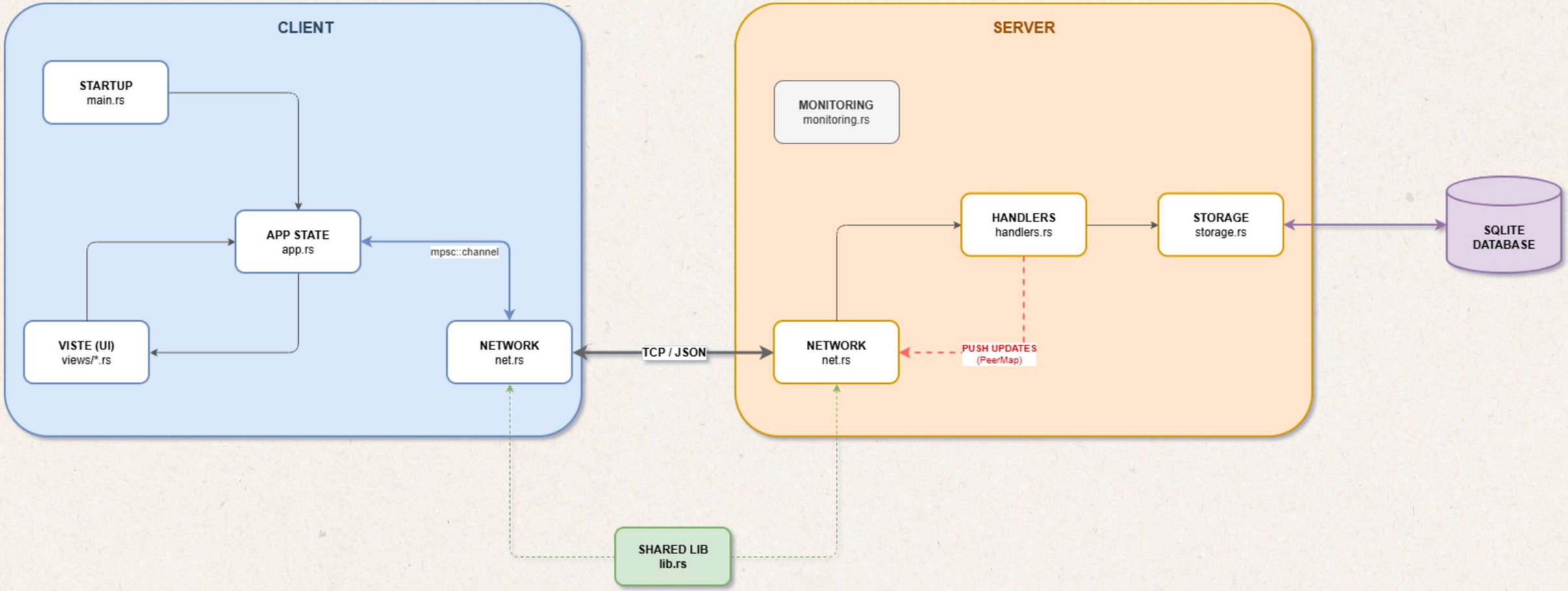
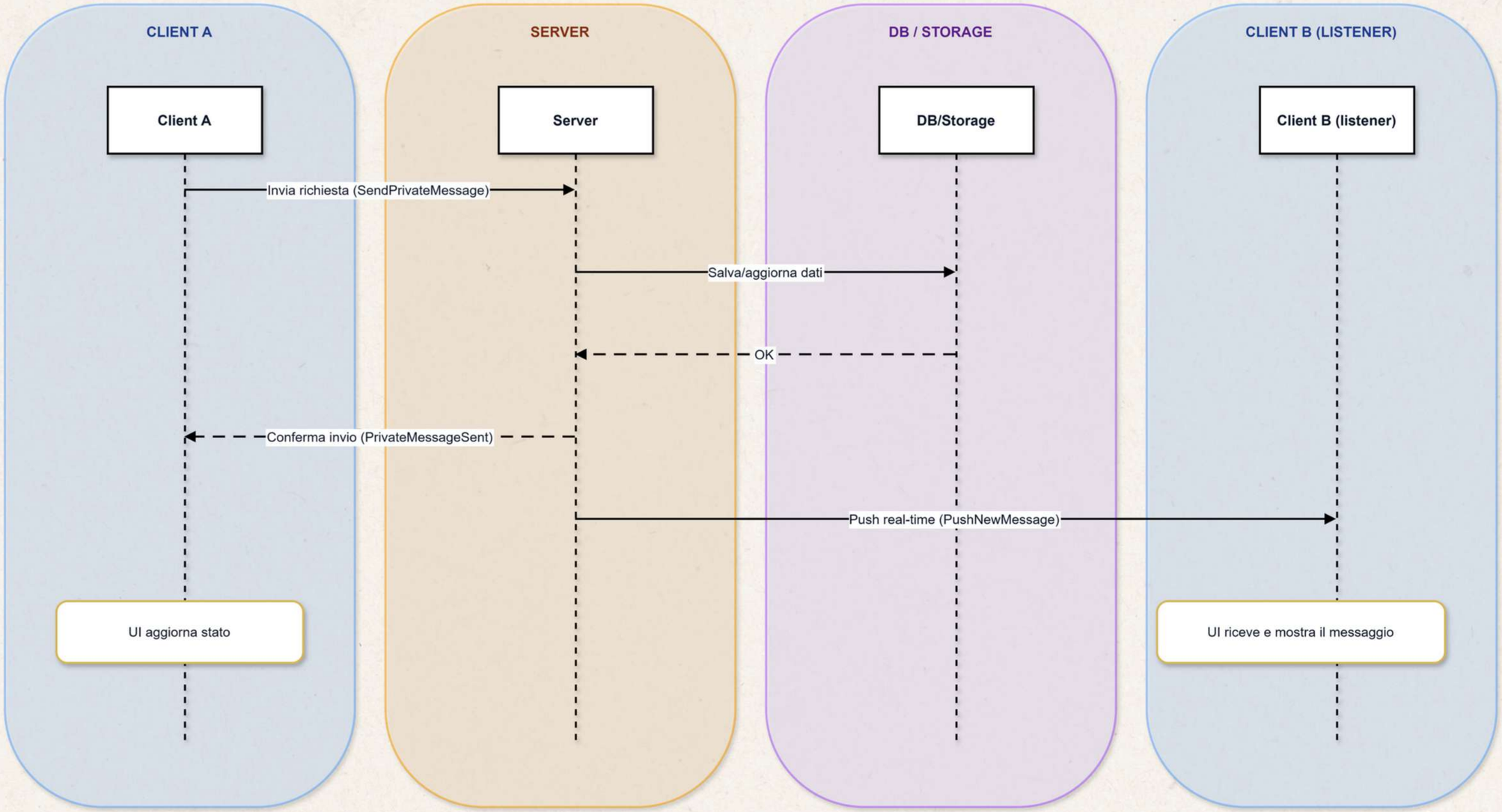
RUGGINE G65

- Centonze Tommaso
- Reale Samuele
- Fuoco Lucio
- Tornesello Gianluca

Architettura generale



Flusso di esecuzione



Shared - lib.rs

Definisce messaggi e tipi condivisi per garantire comunicazione consistente e serializzazione tra client e server.

ClientMsg (Richieste)

- **Autenticazione**
  - Register, Login
- **Chat Privata**
  - StartPrivateChat
  - SendPrivateMessage
  - GetPrivateChats
- **Gruppi e Ricerca**
  - CreateGroup, AddMember...
  - SearchUsers

Modello dati condivisi

- UserInfo
- PrivateChatInfo
- GroupInfo
- Message

ServerMsg (Risposte)

- **Conferme**
  - Registered, LoginOk...
- **Dati Richiesti**
  - UsersFound, PrivateChats
  - GroupMessages
- **Push Realtime ed Errori**
  - PushNewMessage
  - Error

CLIENT

Networking asincrono

net.rs

Application State & UI

app.rs

Il “ponte” Sync/Async

MPSC channel

Application state - UI - app.rs

Il file app.rs contiene la logica principale dell'applicazione sul lato client, che gestisce lo stato dell'applicazione, la navigazione tra le schermate (view), l'interfaccia utente (UI), e la gestione degli eventi.

FUNZIONE CORE:

update()

Ciclo di rendering che ridisegna l'interfaccia grafica frame per frame (60fps)

CRATES PRINCIPALI:

eframe

egui

Il “ponte” Sync/Async - MPSC channel

Il canale MPSC collega in modo sicuro la rete asincrona e la GUI sincrona.

CORE TASKS:

Ricezione dati

Ricezione ad alta velocità
senza bloccare il frame-rate
dell’interfaccia

Isolamento

Isolamento logica di rete dal
rendering grafico

Thread-safe

Iniezione eventi dal task di rete,
senza toccare la memoria della UI

Networking asincrono - net.rs

Gestisce l'intera comunicazione client-server mantenendo attiva una connessione TCP persistente tramite il runtime asincrono di Tokio, occupandosi anche della serializzazione e deserializzazione dei pacchetti dati JSON, trasformando i byte grezzi in strutture dati tipizzate (ServerMsg).

FUNZIONE CORE:

listen_background()

Task in background che mantiene
e gestisce la connessione TCP.

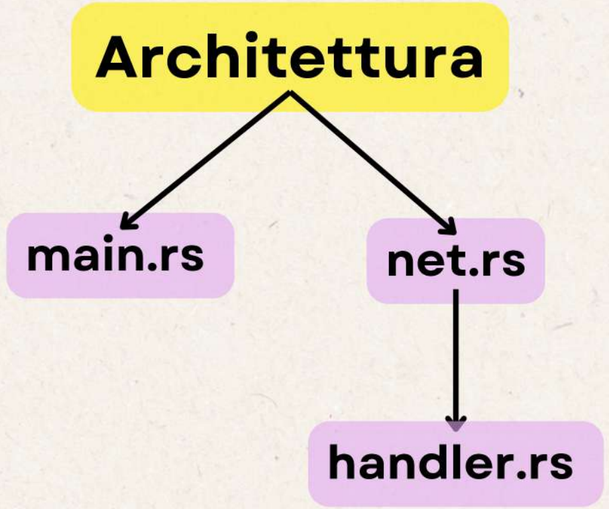
CRATES PRINCIPALI:

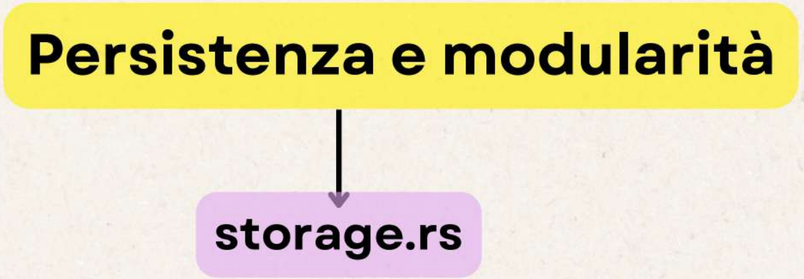
tokio

tokio-util

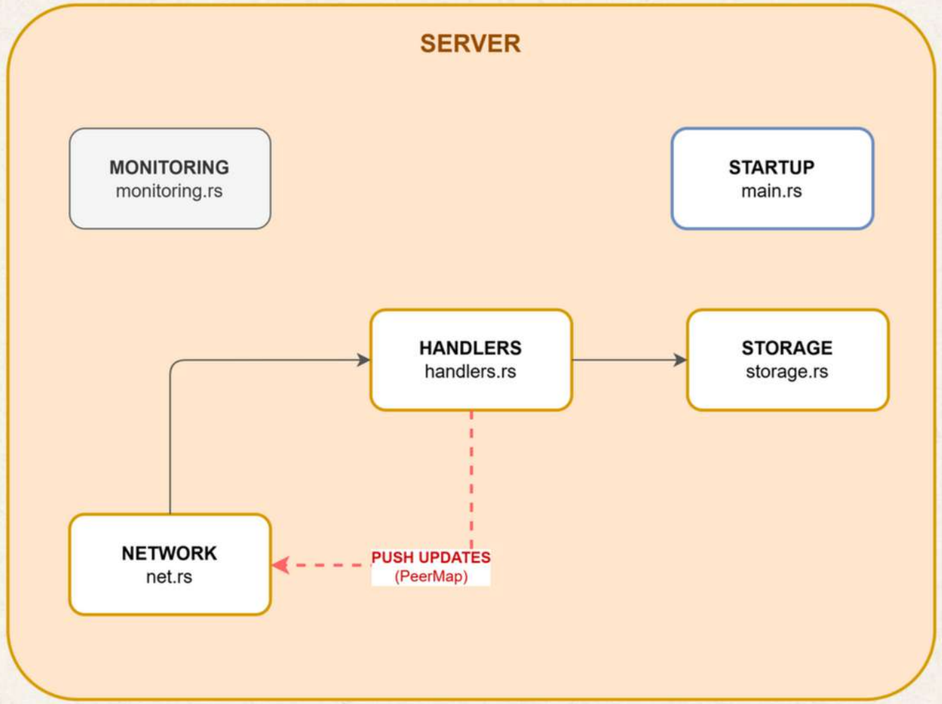
serde_json

SERVER


Architettura
main.rs
net.rs
handler.rs


Persistenza e modularità
storage.rs

Architettura



Main.rs

- Fulcro di funzionamento: **Tokio** (Alta Concorrenza)
- Avvio**: Bootstrap DB**
- Stato Condiviso: **PeerMap**

Net.rs

- Gestisce in tempo reale:
  - Richieste in arrivo dal client
  - Notifiche dal server verso il client per utenti online
- Routing: Dispatcher dei comandi verso gli handlers.rs

Handlers

- Gestisce le richieste in arrivo dal client
- Delega a storage eventuali operazioni fatte sul database
- Controllo in PeerMap se l'utente è online

Persistenza e moduliarità - storage.rs

Per la persistenza dei dati, utilizziamo un database SQLite (ruggine.db), gestito interamente dal modulo storage.rs.

Schema del Database

- Design per Chat Private e Gruppi separati
- Integrità: Messaggio collegato solo a Chat O Gruppo (Vincolo CHECK).

Struttura e inizializzazione

- Modularità: storage.rs isola tutta la logica CRUD dal server
- Avvio Idempotente: fn init crea lo Schema completo all'avvio.

monitoring.rs

Il file monitoring.rs si occupa di **monitorare l'uso della CPU**. Utilizzando la libreria **sysinfo**, il file raccoglie periodicamente informazioni sull'uso della CPU e le registra in un **file di log.**

FUNZIONI UTILIZZATE:

write_log()

Apre o crea un file di log e scrive i dati nel seguente formato:
[YYYY-MM-DD HH:MM:SS] CPU Usage: X.XX%

start_monitoring()

Avvia il **monitoraggio continuo** e registra l'utilizzo CPU ogni intervallo di tempo specificato

STRUMENTI UTILIZZATI:

std::fs::OpenOptions

Per l'apertura del file di log

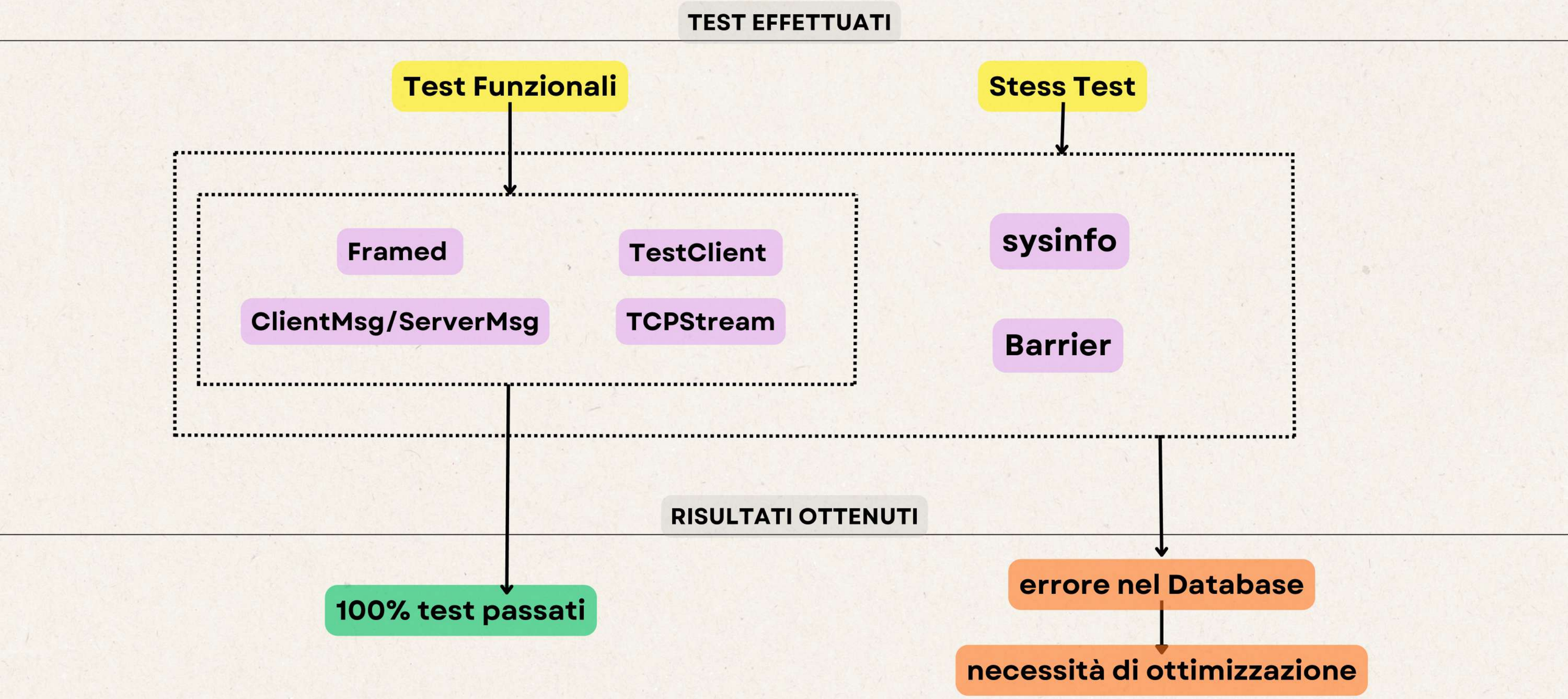
chrono::Local

ottenere l'ora attuale per il formato del file di log

sysinfo

Raccolta delle informazioni di sistema

Test


TEST EFFETTUATI
Test Funzionali
Stess Test
Framed
TestClient
ClientMsg/ServerMsg
TCPStream
sysinfo
Barrier
RISULTATI OTTENUTI
100% test passati
errore nel Database
necessità di ottimizzazione

Scenario in esame

250 utenti che tentano di inviare messaggi simultaneamente per 60 secondi.

Test	Utenti	CPU Media (%)	RAM Max (MB)	Messaggi Inviati	Efficienza (%)
T1	10	14%	26	600 / 600	100%
T2	25	30%	30	1500 / 1500	100%
T3	50	48%	35	3000 / 3000	100%
T4	100	67%	44	6000 / 6000	100%
T5	150	76%	59	8999 / 9000	99%
T6	250	84%	79	9691 / 15000	65%

Ottimizzazioni

PROBLEMI RISCONTRATI

Locking del Database

SOLUZIONI ADOTTATE

pragma_update()

Rollback Journaling

Abilitazione del Write-Ahead Logging

synchronous FULL

busy_timeout = 5000

Scenario POST ottimizzazione

Test	Utenti	CPU Media (%)	RAM Max (MB)	Messaggi Inviati	Efficienza (%)
Test 1	10	7%	26 MB	600 / 600	100%
Test 2	25	25%	30 MB	1500 / 1500	100%
Test 3	50	42%	37 MB	3000 / 3000	100%
Test 4	100	**61%**	48 MB	6000 / 6000	100%
Test 5	150	72%	60 MB	8999 / 9000	99%
Test 6	250	81%	83 MB	15000 / 15000	100%